

ITF23019: Parallel and Distributed Programming

Spring Semester, 2026

Lab 11: Socket Programming

Submission Deadline: April 8th, 2026 23:59

You need at least 50 points to pass this lab assignment!

This lab focuses on intercommunication between processes on the same or different computers in a distributed system. You will learn elements of network programming and how to use sockets for network applications. In addition, you will explore how to use various classes in `java.net` package for creating sockets and message communications with different protocols.

1 Client - Server Communication

In general, a program(s) running on a client machine(s) makes a request to a program (often called as server program) which is running on a server machine. This involves networking services provided by the transport layer in TCP/IP (Transport Control Protocol/Internet Protocol) software stack. The two most popular protocols in the transport layers are TCP (Transport Control Protocol) and UDP (User Datagram Protocol). TCP is a connection-oriented protocol that provides a reliable flow of data between two computers while UDP is a protocol that sends independent packets of data, called datagrams, from one computer to another with no guarantees about arrival and sequencing. The most widely used programming interfaces for these protocols are sockets.

1.1 Socket

Sockets provide an interface for network programming at the transport layer. Network communication using sockets is similar to file input/output (I/O), as socket handles are treated like file handles. The same streams used in file I/O operations also apply to socket-based I/O. Moreover, socket-based communication is independent of the programming language used for implementation. This means a socket program written in Java can communicate with another socket program written in Java or a different language, such as Python, C, or C++ [1].

A server can run multiple services simultaneously, each assigned to a specific port number. When a client requests a particular service, it must identify which server, indicated by a unique IP address, offers the service, and which specific service, indicated by a port number, it is requesting. Thus, a combination of *an IP address* and *a port number* is needed to specify the unique service.

A socket is one endpoint of a two-way communication link between two programs on the network. A client creates a socket and attempts to connect that socket to a server. The socket has two following information:

- IP address

- Port number

A server (program) runs on a specific computer and has a socket that is bound to a specific port. The server listens to the socket for a client to make a connection request (Figure 2a). If everything goes well, the server accepts the connection (Figure 2b).

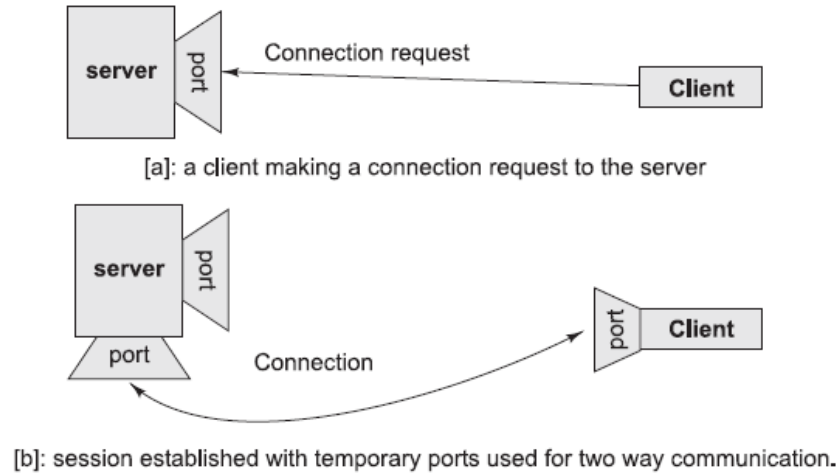


Figure 1: Establishment of path for two-way communication between a client and server[1].

1.2 TCP/IP Socket

TCP socket programming uses `java.net` package. This package supports two key classes for connection-oriented: `ServerSocket` and `Socket`. A server program creates a specific type of socket that is used to listen for client requests (server socket). In the case of a connection request, the program creates a new socket through which it will exchange data with the client using input and output streams. The socket abstraction is very similar to the file concept: developers have to open a socket, perform I/O, and close the socket in the end. Figure 3 illustrates key steps involved in creating socket-based server and client programs[1].

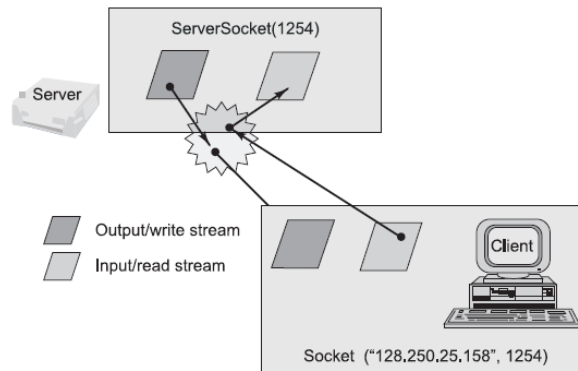


Figure 2: Socket-based client and server programming.

An example of TCP socket is in **echo-server** project. This project has a client program and a server program. You can run these two programs in the same or in different computers with suitable setting (in theory). In this project, the **EchoServer** class implements an echo server. The **EchoClient** class implements a client that connects to the **EchoServer**. The echo server receives data from the client and echoes the data back. The **EchoClient** creates a socket, and thus getting a connection to the echo server. It reads input from the user on the standard input stream, and then forwards that text to the echo server by writing the text to the socket. The server echoes the input back through the socket to the client. The client then reads and displays the data passed back to it from the server. Note that the **EchoClient** both writes to and reads from its socket, thereby sending data to and receiving data from the echo server.

- Client implementation: Use **Socket** class

The following block of code is in the **EchoClient** class. It establishes the socket connection between the client and the server and open a **PrintWriter** and a **BufferedReader** on the socket:

```
String hostName = "localhost"; //127.0.0.1
int portNumber = 6565;

try (Socket echoSocket = new Socket(hostName, portNumber); //1st
    PrintWriter out = new PrintWriter(echoSocket.getOutputStream(), true); //2nd
    BufferedReader in = new BufferedReader(new
        InputStreamReader(echoSocket.getInputStream())); //3rd
    BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in))) //4th
    ...
```

The first line indicates **hostName** or internet address of the Server. **localhost** indicates that the Server program will be run in the local host (You can use 127.0.0.1 to indicate the local host). In this case, client program and server program are run on the same computer. If the server program is running on a different computer, you have to replace "localhost" by the IP address of the server machine (but sometimes I do not get them work!). Note that **hostName** and **portNumber** are information in a socket. Next, the first statement in the **try**-with resources statement creates a new **Socket** object and names it **echoSocket**. The **Socket** constructor requires the name of the computer and the port number to which the client wants to connect. Once the socket is created, it is treated like a file. You can open the file to read the data from or write data to the file. Writing data to the file is similar to sending socket to the Server. Reading data from the file is similar to receiving the socket from the Server. Because the socket is treated like a file, we can use some libraries in Java to read data from or write data to the file. In this project, **PrintWriter** and **BufferedReader** are used. The second statement in the **try** statement gets the socket's output stream and opens a **PrintWriter** on it. Similarly, the third statement gets the socket's input stream and opens a **BufferedReader** on it. The **EchoClient** uses readers and writers so that it can write Unicode characters over the socket. To send data through the socket to the server, the **EchoClient** needs to write to the **PrintWriter**. To get the server's response, **EchoClient** reads from the **BufferedReader** object **stdIn**, which is created in the fourth statement in the **try** statement.

The program has the **while** loop. The loop reads a line at a time from the standard input stream and immediately sends it to the server by writing it to the **PrintWriter** connected to the socket:

```
String userInput;
while ((userInput = stdIn.readLine()) != null) {
    out.println(userInput);
    System.out.println("echo: " + in.readLine());
}
```

In the above code, you can see that in order to send a message to the Server, the Client use **out()** method.

The last statement in the `while` loop reads a line from the `BufferedReader` connected to the socket. The `readLine` method waits until the server echoes the information back to `EchoClient`. When `readLine` returns, `EchoClient` prints the information to the standard output. The `while` loop continues until the user types an end-of-input character. That is, the `EchoClient` reads input from the user, sends it to the Echo server, gets a response from the server, and displays it, until it reaches the end-of-input. (You can type an end-of-input character by pressing Ctrl-C.) The `while` loop then terminates, and the Java runtime automatically closes the readers and writers connected to the socket and to the standard input stream. It closes the socket connection to the server. The Java runtime closes these resources automatically because they were created in the try-with-resources statement. The Java runtime closes these resources in reverse order that they were created. (This is good because streams connected to a socket should be closed before the socket itself is closed.)

This example is straightforward and simple because the echo server implements a simple protocol. The client sends text to the server and the server echoes it back. When your client programs are talking to a more complicated server such as an HTTP server, the client program will be more complicated. However, the basics are much the same as they are in this program:

- Open a socket.

```
Socket echoSocket = new Socket(hostName, portNumber);
```

- Open an input stream and output stream to the socket.

```
PrintWriter out = new PrintWriter(echoSocket.getOutputStream(), true);
BufferedReader in = new BufferedReader(new
    InputStreamReader(echoSocket.getInputStream()));
```

- Read from and write to the stream according to the server's protocol.

```
while ((userInput = stdIn.readLine()) != null) {

    if(userInput.equals("exit"))
        break;
    out.println(userInput);
    System.out.println("echo: " + in.readLine());
}
```

- Close the streams.
- Close the socket.

Only step 3 differs from client to client. The other steps are very similar in most TCP/IP applications.

- Server implementation: Use `ServerSocket` class

For server implementation, `ServerSocket` is used to open the port and start listening to the client with the `accept()` method.

```
int portNumber = 6565;
```

```
System.out.printf("Starting Server at port %d\n", portNumber);
```

```

try (ServerSocket serverSocket = new ServerSocket(portNumber);
    Socket clientSocket = serverSocket.accept();
    PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
    BufferedReader in = new BufferedReader(new
        InputStreamReader(clientSocket.getInputStream()));) {
    String inputLine;
    while ((inputLine = in.readLine()) != null) {
        out.println(inputLine);
    }
    ....
}

```

The server is running its service at the port number 6565 (you can change this number to whatever number you want except the ones which are reserved for well-known applications). No IP address is needed here as server is offering the service. Because the port number is unique, server cannot run any other service at the port 6565. Next, the server calls `accept()` method to wait for the request from the client. This is the *blocking* call to receive the request from the Client. The Server is blocked until there is a request from the client. Once there is the client request, the `accept()` returns the socket which can be treated as a file. The server then will perform I/O operations with the client with `out` is to send and `in` is to receive message from the Client.

- Open a socket.

```
ServerSocket serverSocket = new ServerSocket(portNumber);
```

- Wait for the Client Request

```
Socket clientSocket = serverSocket.accept();
```

- Create I/O streams for communicating to the client

```
PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
BufferedReader in = new BufferedReader(new
    InputStreamReader(clientSocket.getInputStream()));
```

- Perform communication with client.

```
while ((inputLine = in.readLine()) != null) {
    out.println(inputLine);
}
```

- Close the socket. The implementation ensures that the `serverSocket` and `clientSocket` are automatically closed because they are inside a `try- block`.

1.3 UDP Socket

TCP guarantees the delivery of packets and packet ordering on destination. However, for some applications, this feature is not required but delay is more important. Thus, it would be better to use a lighter transport protocol and UDP is used for this purpose. While TCP uses stream, UDP uses datagram for a connectionless packet delivery service. Java supports datagram communication through the following classes:

- DatagramPacket
- DatagramSocket

The DatagramPacket class can be used for creating a packet object. The DatagramSocket class supports various methods that can be used for transmitting or receiving data a datagram over the network including `send()` and `receive()` methods.

One example of using UDP socket is `socketUPD` project. This project implements server and client in the `Server` and `Client` classes, respectively.

- Server implementation:

The server program is setup on a designated port:

```
int server_port = 9000;
DatagramSocket serverSocket = new DatagramSocket(server_port);
```

By using DatagramSocket, the socket is set up on the local host. Then, the server prepares a DatagramPacket, which is a packet delivery service designed to deliver packages between computers. This is done by creating a new DatagramPacket and defining its buffer size.

```
byte[] receiveData = new byte[10];
DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
```

The server will then wait until a packet is received. The receive method will block until a packet is received.

```
serverSocket.receive(receivePacket);
```

After receiving a packet, the Server will retrieve the IP address and port number of the sender of the packet.

```
InetAddress IPAddress = receivePacket.getAddress();
int client_port = receivePacket.getPort();
```

The Server then creates another DataPacket, which it will send to the connected Client. This DataPacket contains the message "Welcome from server".

```
sentence = "Welcome from server" ;
byte[] sendData = sentence.getBytes();
DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress,
    client_port);
serverSocket.send(sendPacket);
```

The IPAddress and client port are retrieved from the packet sent from the Client.

- Client implementation:

The Client is set up on a similar fashion to the server, by using the DatagramSocket constructor. However, the port number is not defined, therefore a port will be assigned.

```
DatagramSocket clientSocket = new DatagramSocket();
```

The Client finds the IP address and port number of the server. This is done in line 31 and 32 by using the InetAddress class.

```
InetAddress IPAddress = InetAddress.getByName("localhost");
int port = 9000;
```

The Client generates a DatagramPacket with a message which it sends to the IP address and port number of the server.

```
String sentence = "Hello world";
byte[] sendData = sentence.getBytes();
DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length, IPAddress, port);
clientSocket.send(sendPacket);
```

The Client then prepares to receive a packet in return. This is done in the same way as the Server did. The client creates a DatagramPacket which allocates the space to hold the incoming message and the amount of bytes to read. The DatagramPacket is filled with the incoming message. The message is converted to a string and written to standard out, respectively.

```
byte[] receiveData = new byte[19];
DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
// The Client will wait untill a package is received.
clientSocket.receive(receivePacket);
String receivedSentence = new String(receivePacket.getData());
System.out.println("Received from server: " + receivedSentence);
```

2 Multicast Communication

2.1 IP multicast

IP multicast is built on top of the Internet Protocol (IP). Note that IP packets are addressed to computers – ports belong to the TCP and UDP levels. IP multicast allows the sender to transmit a single IP packet to a set of computers that form a multicast group. The sender is unaware of the identities of the individual recipients and of the size of the group. A multicast group is specified by a Class D Internet address (shown in Figure 4) – that is, an address whose first 4 bits are 1110 in IPv4 address[2].

	octet 1	octet 2	octet 3	Range of addresses
	Network ID		Host ID	
Class A:	1 to 127	0 to 255	0 to 255	1.0.0.0 to 127.255.255.255
	Network ID		Host ID	
Class B:	128 to 191	0 to 255	0 to 255	128.0.0.0 to 191.255.255.255
	Network ID		Host ID	
Class C:	192 to 223	0 to 255	0 to 255	192.0.0.0 to 223.255.255.255
			1 to 254	
Class D (multicast):	224 to 239	0 to 255	0 to 255	224.0.0.0 to 239.255.255.255
	Multicast address			
Class E (reserved):	240 to 255	0 to 255	0 to 255	240.0.0.0 to 255.255.255.255
			1 to 254	

Figure 3: Socket-based client and server programming.

Being a member of a multicast group allows a computer to receive IP packets sent to the group. The membership of multicast groups is dynamic, allowing computers to join or leave at any time and to join an

arbitrary number of groups. It is possible to send datagrams to a multicast group without being a member.

At the application programming level, IP multicast is available only via UDP. **An application program performs multicasts by sending UDP datagrams with multicast addresses and ordinary port numbers.** It can join a multicast group by making its socket join the group, enabling it to receive messages to the group. At the IP level, a computer belongs to a multicast group when one or more of its processes has sockets that belong to that group. When a multicast message arrives at a computer, copies are forwarded to all of the local sockets that have joined the specified multicast address and are bound to the specified port number[2].

Datagrams multicast over IP multicast have the same failure characteristics as UDP datagrams – that is, they suffer from omission failures. The effect on a multicast is that messages are not guaranteed to be delivered to any particular group member in the face of even a single omission failure. That is, some but not all of the members of the group may receive it. This can be called unreliable multicast, because it does not guarantee that a message will be delivered to any member of a group.

2.2 Java API to IP multicast

Refer to `multicast` project. This project consists of two programs: one for sender and the other for receiver. The project illustrate how multiple receivers can receive a message sent to a group.

- Receiver implementation

First, multicast is based on the concept of group membership, where a multicast address represents each group. Class D addresses (that is, addresses in the range 224.0.0.0 to 239.255.255.255) are reserved for multicast traffic and managed globally by the Internet Assigned Numbers Authority (IANA). A receiver must subscribe or join the group to become a member. Only those that subscribe to a group receive packets sent to that group.

The Java API provides a datagram interface for IP multicast through the *MulticastSocket* class, which is a subclass of *DatagramSocket* with the additional capability of joining multicast groups. The *MulticastSocket* class offers two alternative constructors, allowing sockets to be created with either a specified local port or any available local port.

A process can join a multicast group with a given multicast address by invoking the *joinGroup* method on its multicast socket. After binding the *MulticastSocket* to a port, the *joinGroup()* method is called with the multicast IP as an argument. This step is necessary to receive packets sent to the group.

```
String multicastAddress = "224.0.0.3";
int port = 6565;
MulticastSocket socket = new MulticastSocket(port);
InetAddress group = InetAddress.getByName(multicastAddress);
socket.joinGroup(group);
```

The socket joins a multicast group at a given port and will receive datagrams sent by processes on other computers to that group at that port.

```
byte[] buffer = new byte[256];
DatagramPacket packet = new DatagramPacket(buffer, buffer.length);
socket.receive(packet);
```

A process can leave a specified group by invoking the *leaveGroup* method of its multicast socket.

```
socket.leaveGroup(group);
```

- Sender implementation

The sender creates a *DatagramSocket* socket with multicast address dedicated to a group and a selected port.

```
String multicastAddress = "224.0.0.3";  
int port = 6565;  
DatagramSocket socket = new DatagramSocket();
```

The server then creates a *DatagramPacket* packet and finally sends the packet. All receivers of the group will receive that packets.

```
DatagramPacket packet = new DatagramPacket(buffer, buffer.length, group, port);  
socket.send(packet);
```

- Running multicast project

It is meaningful to create one process that sends a message and multiple processes that receive it. If you are using an IDE other than IntelliJ, you can run `MulticastSender.java` once and `MulticastSender.java` multiple times. If you are using IntelliJ, you can create multiple receiving processes by modifying the Run Configuration as follows (refer to Lab 10).

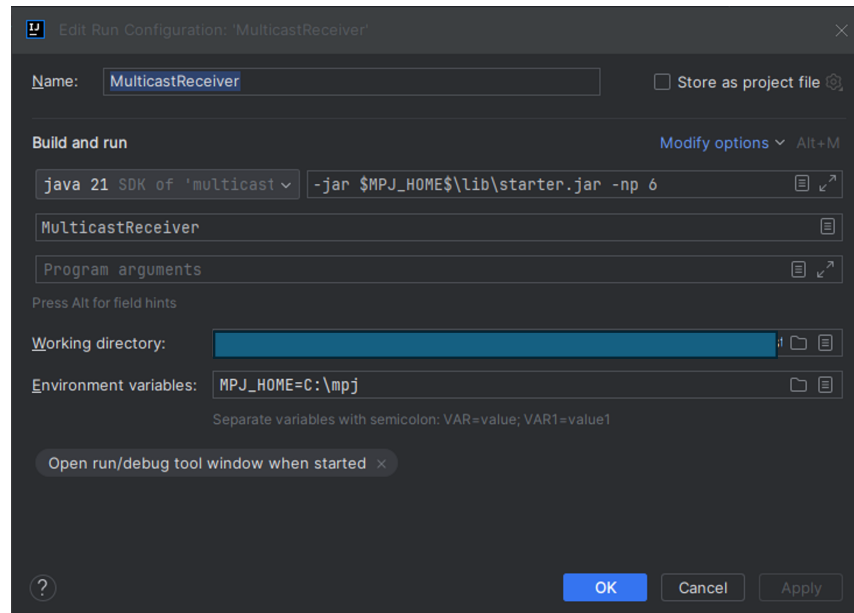


Figure 4: Configuration to create multiple receiver processes.

```

MPJ Express (0.44) is started in the multicore configuratio
Receiver listening for multicast messages...
Receiver listening for multicast messages...
Receiver listening for multicast messages...
Receiver listening for multicast messages...
Receiver listening for multicast messages...
Receiver listening for multicast messages...
Receiver received: Hello, This is a multicast message!
Receiver received: Hello, This is a multicast message!
Receiver received: Hello, This is a multicast message!
Receiver received: Hello, This is a multicast message!
Receiver received: Hello, This is a multicast message!
Receiver received: Hello, This is a multicast message!
Process finished with exit code 0

```

Figure 5: A possible output from receivers.

```

Message sent to multicast group: 224.0.0.3

Process finished with exit code 0

```

Figure 6: Output from sender.

3 Exercises

3.1 Exercise 1 (50 points)

Refer to `socketTCP` project for this exercise.

Answer the following questions from the output of the program:

- Run the code and include the output in your report.
- Which class should run first?
- Which port number is used for Client side and which port number is used for Server side?
- What are the IP address or hostname of Client and Server?

Analyse code for Server and answer the following questions:

- Which line the Server opens the socket?
- Which line the Server waits for Client?
- Which lines the Server creates I/O Streams for communication with the Client?
- Which lines the Server performs communication with the Client?
- Which line the connection is closed?

Analyse code for Client and answer the following questions:

- Which line the Client creates a socket object?
- Which lines the Client creates I/O Streams for communication with the Client?
- Which lines the Client performs communication with the Client?
- Which line the connection is closed?

Finally:

- How do you relate the above operations of Client and Server to the **Three-way handshake** data transmission in TCP protocol?

3.2 Exercise 2 (25 points)

Refer to `socketUDP` project for this exercise.

- Run the code and include the output in your report.
- Which class should run first?
- Create the output similar to the figure below (except the red and green boxes and texts). The project has two programs. **How** and **why** the server can receive multiple requests with multiple ports from client(s)?

```
Server port
Waiting for client on port 9802
Received from client: Hello world
Message sent from IP address: /127.0.0.1 and port: 51106
Received from client: Hello world
Message sent from IP address: /127.0.0.1 and port: 61165
Received from client: Hello world
Message sent from IP address: /127.0.0.1 and port: 61166
Received from client: Hello world
Message sent from IP address: /127.0.0.1 and port: 54959
```

Client ports

Figure 7: One possible output of `socketUDP` project.

3.3 Exercise 3 (25 points)

Refer to `multicast` project for this exercise.

- Reproduce Figures 6 and 7.
- Can we setup multiple senders in the same way with setting multiple receivers? Explain why and why not.

4 What To Submit

Complete the the exercises in this assignment. Then, put your lab11 report file (Markdown, TXT or PDF file) inside the lab11 directory of your repository. Commit your changes and remember to check the GitHub website to make sure all files have been submitted.

5 References

1. Rajkumar Buyya et al., Object-Oriented Programming with Java, Chapter 13.
2. Textbook